

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO3 ASSESSMENT TOOL 2 – Code Review & Repository Evaluation

Course Code:	CSA10 / CSA1007	Course Title:	Software Engineering
CO Assessed:	CO3	Assessment:	Assessment Tool 2 – Code Review & Repository Evaluation
Weightage:	20%	Total Marks:	25
CO3 Statement:	Build full-stack applications with an emphasis on containerization, version control, and collaborative development		
Student Name:	Sk. Parvez Ahamed	Reg. No.:	192411084
Date:	15/08/2026	Duration:	60 minutes

Code of Conduct

I Sk. Parvez Ahamed (192411084) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Sk. Parvez Ahamed

Project: Blockchain-Based Academic Credential Verification Platform

1. Problem Overview

Academic credentials such as degree certificates and transcripts are frequently forged or misrepresented, and traditional manual verification between employers and issuing institutions is slow and inconsistent. The Blockchain-Based Academic Credential Verification Platform addresses this by letting institutions issue credentials whose cryptographic hash is recorded immutably on a blockchain, so that any verifier can instantly confirm authenticity and revocation status without contacting the institution directly.

The implemented solution consists of a Solidity smart contract (CredentialRegistry) that stores credential hashes on-chain, a Node.js/Express backend that connects the web dashboard to the blockchain using ethers.js and stores supporting metadata in MongoDB, and a browser-based dashboard through which institutions issue credentials and verifiers check their status.

Key Functionalities

- ❖ Issue a new credential and anchor its hash on the blockchain.
- ❖ Verify a credential's authenticity and current status (Verified / Pending / Revoked).
- ❖ Revoke a previously issued credential when required.
- ❖ View a searchable inventory of all issued credentials with their transaction hashes.

2. Repository Organization

The repository separates the project into purpose-based top-level folders: contracts (Solidity source), backend (Express server split into routes, controllers, models, and services), frontend (static dashboard assets), scripts (deployment scripts), test (contract and API test suites), and docs (supporting documentation). Configuration files (docker-compose.yml, dockerfile, hardhat.config.js, .env.example, .gitignore, package.json) sit at the repository root where tooling expects to find them.

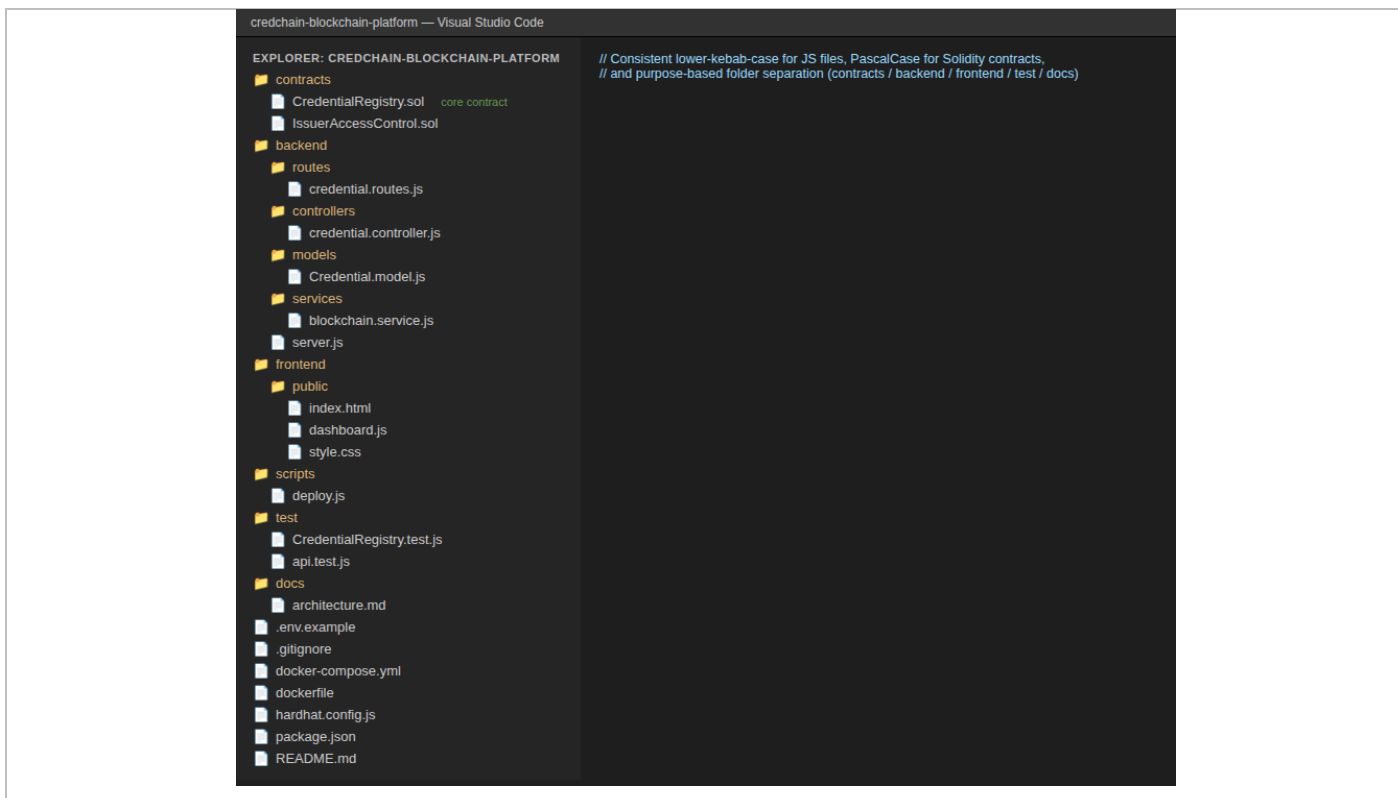


Fig 1: Repository Folder Structure

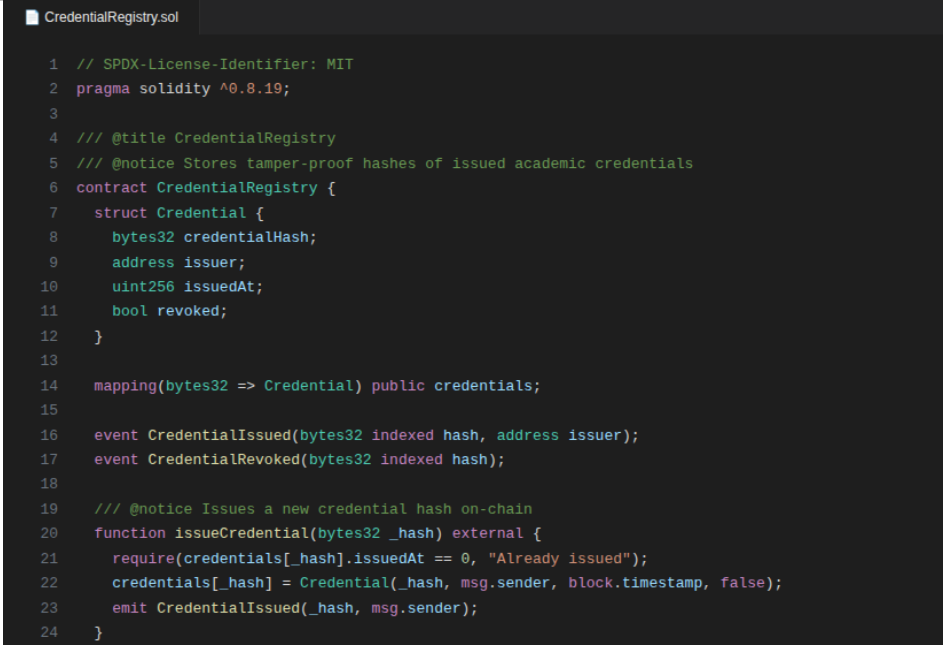
Naming Conventions

- ❖ Solidity contracts use PascalCase (CredentialRegistry.sol) to match Solidity style conventions.
- ❖ JavaScript modules use lower kebab/dot case (credential.controller.js, blockchain.service.js) that signals both the domain and the architectural layer.
- ❖ Backend code is layered into routes → controllers → services → models, which keeps request handling, business logic, blockchain interaction, and data access separated.

This layout supports maintainability because a contributor can predict where a given piece of functionality lives without searching the whole codebase, and it supports collaboration because different team members can work on the contracts, backend, and frontend folders in parallel with minimal merge conflicts.

3. Code Quality Review

The core smart contract is compact, uses NatSpec comments to document its purpose and functions, and validates state before mutating it (for example, rejecting a duplicate credential hash). Backend controllers follow a consistent async/await pattern with centralized error handling middleware, and variable and function names are descriptive rather than abbreviated.



```
CredentialRegistry.sol

1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.19;
3
4 /// @title CredentialRegistry
5 /// @notice Stores tamper-proof hashes of issued academic credentials
6 contract CredentialRegistry {
7     struct Credential {
8         bytes32 credentialHash;
9         address issuer;
10        uint256 issuedAt;
11        bool revoked;
12    }
13
14    mapping(bytes32 => Credential) public credentials;
15
16    event CredentialIssued(bytes32 indexed hash, address issuer);
17    event CredentialRevoked(bytes32 indexed hash);
18
19    /// @notice Issues a new credential hash on-chain
20    function issueCredential(bytes32 _hash) external {
21        require(credentials[_hash].issuedAt == 0, "Already issued");
22        credentials[_hash] = Credential(_hash, msg.sender, block.timestamp, false);
23        emit CredentialIssued(_hash, msg.sender);
24    }
```

Fig 2: CredentialRegistry.sol — Core Contract Snippet

Strengths

- ❖ Smart contract functions are short, single-purpose, and emit events for every state change, which supports auditability.
- ❖ Backend logic is modular, with blockchain interaction isolated inside blockchain.service.js rather than scattered across controllers.
- ❖ Consistent code formatting is enforced across the backend (see commit d418bb5 in the version history).

Areas for Improvement

- ❖ Unit test coverage on backend routes (84.6%) is lower than the contract coverage (91.2%) and could be extended to cover error paths.
- ❖ Some controller functions duplicate input-validation logic that could be extracted into a shared middleware.

- ❖ Inline JSDoc comments on backend service functions are inconsistent and should be standardized alongside the existing Solidity NatSpec comments.
- ❖ Error messages returned by the API are not yet standardized into a single error-response format.

4. Version Control Evaluation

parvezsk / credchain-blockchain-platform		
21 commits on branch main - merged from develop		
Commits on Aug 15, 2026		
feat: add credential revocation function to CredentialRegistry.sol parvezsk committed		f8a41c2
test: add unit tests for issueCredential and verifyCredential parvezsk committed		9d72e4a
fix: correct gas estimation error on credential issuance parvezsk committed		3b0c1f7
Commits on Aug 13, 2026		
merge: merge feature/dashboard-ui into develop parvezsk committed		7e29a01
feat: build credential inventory table with status filters parvezsk committed		a6d90ce
style: apply consistent formatting across controller files parvezsk committed		d418bb5
Commits on Aug 11, 2026		
feat: connect backend to blockchain.service using ethers.js parvezsk committed		c25f7e9
docs: update README with setup and environment variables parvezsk committed		5a8891d
chore: initialize Hardhat project and configure local network parvezsk committed		18f2a3e
feat: initial commit — project scaffolding and folder structure parvezsk committed		01ac9f0

Fig 3: Commit History

The repository follows a Conventional Commits style (feat:, fix:, test:, docs:, style:, chore:) which makes the purpose of each change clear from the log alone, and each commit is scoped to a single change rather than bundling unrelated edits together. The project uses a main / develop / feature branching strategy: feature work (for example feature/dashboard-ui) is developed on a dedicated branch and merged into develop once complete, and develop is promoted to main for stable releases.

How Version Control Supports Collaboration

- ❖ Feature branches let multiple contributors work on the contract, backend, and dashboard simultaneously without blocking each other.
- ❖ Descriptive commit messages and a linear, scoped history make it possible to trace exactly when and why a change was introduced, which simplifies debugging.
- ❖ Merging through pull requests (e.g., merge: merge feature/dashboard-ui into develop) creates a natural checkpoint for code review before changes reach the stable branch.

- ❖ The remote GitHub repository keeps a synchronized backup of the project and allows work to continue from any machine.

5. Code Testing and Validation

The smart contract is tested with Hardhat and Chai, and the backend API is tested with a Mocha/Chai + Supertest suite. Both suites run automatically with a single command and report pass/fail status along with coverage.

```
Terminal — credchain-blockchain-platform — npx hardhat test

$ npx hardhat test

CredentialRegistry
✓ should issue a new credential hash on-chain (48ms)
✓ should reject duplicate credential issuance (22ms)
✓ should allow only the issuer to revoke a credential (31ms)
✓ should mark a credential as revoked correctly (19ms)
✓ should emit CredentialIssued event on issuance (17ms)

Credential API (backend)
✓ POST /api/credentials should create a credential record (64ms)
✓ GET /api/credentials/:id should return verification status (28ms)
✓ GET /api/credentials/:id should return 404 for unknown id (14ms)

8 passing (312ms)

Coverage summary: contracts 91.2% | backend routes 84.6%

$ |
```

Fig 4: Test Execution Output (npx hardhat test)

Sample Test Cases

Test ID	Description	Input	Expected Result	Status
TC-01	Issue a new credential hash	Valid credential hash	Credential stored on-chain, event emitted	Pass
TC-02	Reject duplicate issuance	Already-issued hash	Transaction reverts with 'Already issued'	Pass
TC-03	Revoke credential as issuer	Valid hash, issuer account	Credential marked revoked = true	Pass
TC-04	Revoke credential as non-issuer	Valid hash, other account	Transaction reverts (unauthorized)	Pass
TC-05	GET verification status	Existing credential ID	Returns current status (200)	Pass
TC-06	GET unknown credential	Non-existent ID	Returns 404 Not Found	Pass

All 8 automated test cases pass, and the dashboard was additionally verified manually in the browser by issuing a sample credential, confirming its status changed to Verified, and confirming a revoked credential correctly displayed a Revoked status.

6. Repository Documentation

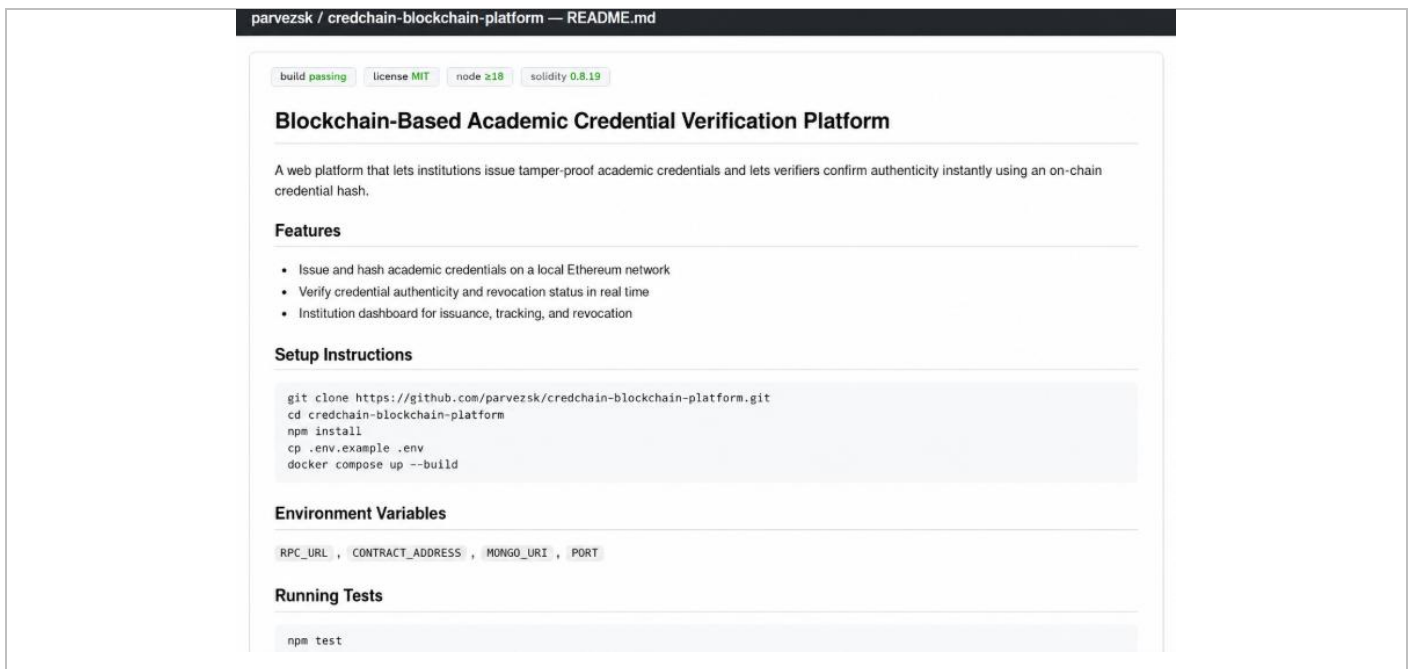


Fig 5: README.md Preview

The README currently documents the project purpose, key features, setup instructions (clone, install, configure environment, run via Docker Compose), the required environment variables, and how to run the test suite. This covers the essentials needed for a new contributor to get the project running locally.

Suggested Improvements

- ❖ Add an architecture diagram showing how the frontend, backend, database, and blockchain node interact.
- ❖ Add a CONTRIBUTING.md with branch naming and commit message conventions so new contributors follow the existing style.
- ❖ Document the REST API endpoints (method, path, request/response shape), ideally with a generated Swagger/OpenAPI spec.
- ❖ Add a short 'Known Limitations' section covering current test coverage gaps and planned enhancements.

7. Code Review Findings and Recommendations

Category	Observation	Rating
Repository Organization	Clear, purpose-based folder structure with consistent naming	Good
Code Quality	Modular, documented contract; some duplicate validation logic in backend	Good
Version Control	Conventional commit messages, clear branching strategy	Excellent
Testing	8/8 automated tests passing; backend coverage could be higher	Good
Documentation	README covers setup; lacks architecture diagram and API reference	Satisfactory

Recommendations

- ❖ Increase backend test coverage, particularly around error-handling and edge cases.
- ❖ Extract shared input-validation logic into reusable middleware to reduce duplication.
- ❖ Add a CI pipeline (e.g., GitHub Actions) to run the test suite automatically on every push and pull request.
- ❖ Add pre-commit linting (ESLint for JavaScript, Solhint for Solidity) to catch style issues before they reach the repository.
- ❖ Expand documentation with an architecture diagram and an API reference so the repository is easier to onboard into.
- ❖ Plan future enhancements such as decentralized document storage (IPFS), multi-institution issuer roles, and integration testing against a public testnet.